

Final Phase Documentation

1. Project Overview

TaxIt is a Kotlin Android application that helps users calculate sales tax for a purchase based on a ZIP code. The user enters a ZIP code, adds one or more line items, enters quantity and price, optionally adds a discount, and then calculates the tax and final total.

The app uses the Ziptax API to look up sales tax information for the ZIP code. It displays the city/state, state tax, county tax, city tax, and combined tax rate. The app also saves completed calculations to a local Room database so users can view their calculation history later.

TaxIt uses tools such as Kotlin, MVVM architecture, Hilt dependency injection, Retrofit/OkHttp, Room database, StateFlow, RecyclerView, and Material UI components.

2. Final Storyboard

Screen 1: Calculator Screen

Purpose:

The Calculator screen is the main screen where the user enters purchase information and calculates sales tax.

Main elements:

- ZIP code input
- Tax rate lookup status
- Tax breakdown card
- Line item list
- Add Item button
- Discount percentage input
- Subtotal, discount, tax, and total display
- Calculate Tax button
- Clear All button

How the user interacts with it:

The user enters a ZIP code such as 10001. The app looks up the tax rate using Ziptax and displays the city, state, and tax breakdown. The user then enters item details such as description, quantity, and price. If needed, the user can add a discount percentage. When the user taps Calculate Tax, the app calculates the discounted taxable amount, tax amount, and final total.

Annotated wireframe:

[Top dark total panel]

- Subtotal: shows the total before discount and tax
- Discount: shows the discount amount subtracted from subtotal

- Tax: shows the calculated tax amount
- Total: shows the final amount after discount and tax

[ZIP lookup section]

- ZIP field: user enters 5-digit ZIP code
- Status text: shows loading, success, or error message
- Tax breakdown card: displays state, county, city, and combined tax rates

[Line item section]

- Description: item name or service name
- Quantity: number of units
- Price: item price
- X/delete: removes that item row
- Add Item: adds another row

[Discount section]

- Discount %: applies one discount to the whole order

[Bottom buttons]

- Calculate Tax: performs the final calculation and saves it
- Clear All: resets the calculator

Screen 2: Tax Rate Breakdown

Purpose:

This section explains the tax rate for the ZIP code.

Main elements:

- City and state summary
- State tax rate
- County tax rate
- City tax rate
- Combined tax rate

Example:

For ZIP 10001, the app can display:

- New York City, NY
- State: 4.00%
- County: 0.00%
- City: 4.50%
- Combined Rate: 8.88%

How the user interacts with it:

The user does not need to manually enter the tax rates. The app automatically receives them from the Ziptax API after the ZIP code lookup succeeds.

Screen 3: History Screen

Purpose:

The History screen displays previously saved tax calculations.

Main elements:

- History title
- Clear All button
- Saved calculation cards
- Empty history message when no calculations exist

Each history card shows:

- ZIP code and city/state
- Date and time of calculation
- Subtotal
- Tax
- Total
- X/delete button

How the user interacts with it:

After calculating tax, the user can go to the History tab and see the saved calculation. The user can delete one saved calculation by tapping the X on that card. The user can also clear all saved history by tapping Clear All.

Screen 4: Clear All Confirmation Dialog

Purpose:

This dialog prevents the user from accidentally deleting all saved history.

Dialog message:

Clear all history?

This will permanently delete every saved calculation.

Buttons:

- Cancel
- Clear All

How the user interacts with it:

If the user taps Cancel, nothing is deleted. If the user taps Clear All, all saved history records are removed from the Room database.

Screen 5: Empty History Screen

Purpose:

This screen appears when there are no saved calculations.

Message:

No saved calculations yet. Run a tax calculation to see it here.

How the user interacts with it:

The user can return to the Calculator screen, run a calculation, and then the result will appear in History.

3. Feature Explanation

Feature 1: ZIP Code Tax Lookup

The user enters a 5-digit ZIP code. The app sends that ZIP code to the Ziptax API through Retrofit and OkHttp. Ziptax returns location and tax rate information.

The returned data includes:

- ZIP code
- City
- State
- State sales tax
- County sales tax
- City sales tax
- District/special tax
- Combined sales tax rate

The repository maps the Ziptax response into the app's local cached ZIP rate model. The app then displays the information on the Calculator screen.

If the lookup is loading, the Calculate Tax button is disabled so the user cannot spam the button. If there is an error, the app shows a friendly message instead of a technical error.

Example error message:

Couldn't fetch the tax rate. Check your connection and try again.

Feature 2: Tax Breakdown Display

After a successful ZIP lookup, the app displays the tax breakdown.

Example:

State: 4.00%

County: 0.00%

City: 4.50%

Combined Rate: 8.88%

This helps the user understand where the tax comes from instead of only seeing one final tax number.

Feature 3: Line Items

The user can enter multiple purchase items. Each item includes:

- Description
- Quantity
- Price

The app uses a RecyclerView and LinelItemAdapter to display the item rows. The ViewModel stores the actual line item data, so the items stay available when the user navigates away and returns to the Calculator screen.

The calculation for each item is:

line total = quantity × price

Example:

Description: Car Detail

Quantity: 2

Price: \$100.00

Line total: \$200.00

Feature 4: Add Item

The Add Item button adds another line item row. This allows the user to calculate tax for multiple products or services in one total.

Example:

Item 1: Car Detail

Item 2: Wax Service

Item 3: Interior Cleaning

The ViewModel updates the list, and the adapter refreshes the visible rows.

Feature 5: Delete One Line Item

Each line item row has an X/delete option. When the user taps it, that item is removed from the ViewModel and adapter list.

If the user deletes the last item, the app automatically leaves one blank row so the user always has a place to enter a new item.

Feature 6: Discount Percentage

The app includes a whole-order discount field. The user can enter a discount percentage such as 10.

The discount applies to the subtotal before tax is calculated.

Formula:

subtotal = sum of line items

$\text{discount amount} = \text{subtotal} \times \text{discount percent} / 100$

$\text{taxable amount} = \text{subtotal} - \text{discount amount}$

$\text{tax} = \text{taxable amount} \times \text{combined tax rate}$

$\text{total} = \text{taxable amount} + \text{tax}$

Example:

Subtotal: \$200.00

Discount: 10%

Discount Amount: \$20.00

Taxable Amount: \$180.00

Tax Rate: 8.875%

Tax: \$15.98

Total: \$195.98

The discount input is clamped between 0 and 100.

- Negative values become 0
- Values over 100 become 100
- Blank input is treated as 0
- Invalid input is treated as 0

Feature 7: Calculate Tax

When the user taps Calculate Tax, the app calculates:

- Subtotal
- Discount amount
- Tax amount
- Final total

The price fields are also formatted as currency after calculation.

Example:

100 becomes \$100.00

The calculation is saved to local history using Room.

Feature 8: Clear All Calculator

The Clear All button resets the Calculator screen.

It clears:

- ZIP input
- ZIP lookup state
- Tax breakdown display
- Line items
- Discount input
- Subtotal
- Tax

- Total

After clearing, the user can start a new calculation.

Feature 9: Save to History

After a successful calculation, the app saves the result to the Room database.

Saved information includes:

- ZIP code
- City
- State
- Combined tax rate
- Subtotal
- Tax amount
- Total
- Timestamp

The History screen reads from Room and displays saved calculations.

Feature 10: History Screen

The History screen shows saved calculations as cards.

Each card displays:

- ZIP/city/state
- Date and time
- Subtotal
- Tax
- Total
- Delete X button

This lets the user review previous tax calculations.

Feature 11: Delete One History Card

Each history card has an X button. Tapping it deletes only that specific calculation from the Room database.

Flow:

User taps X

HistoryAdapter sends callback

HistoryFragment tells HistoryViewModel

HistoryViewModel calls TaxRepository

TaxRepository calls DAO

Room deletes the row

History list updates automatically

Feature 12: Clear All History

The History screen has a Clear All button. This deletes all saved calculation records.

Because this is destructive, the app shows a confirmation dialog first.

Flow:

User taps Clear All

Confirmation dialog appears

User taps Cancel or Clear All

If confirmed, all history records are deleted

Feature 13: Empty History Message

If there are no saved calculations, the History screen displays:

No saved calculations yet. Run a tax calculation to see it here.

This makes the screen clearer instead of looking broken or empty.

Navigation Flow 1: Basic Calculation

1. User opens the app.
2. The Calculator screen appears.
3. User enters ZIP code.
4. App looks up tax rate using Ziptax.
5. User enters item description, quantity, and price.
6. User optionally enters a discount percentage.
7. User taps Calculate Tax.
8. App displays subtotal, discount, tax, and total.
9. App saves the calculation to history.

Navigation Flow 2: Viewing History

1. User completes a tax calculation.
2. User taps the History tab.
3. History screen displays saved calculation cards.
4. User can review previous subtotal, tax, total, and date/time.

Navigation Flow 3: Deleting One History Record

1. User goes to History.
2. User taps X on one card.
3. App deletes that one saved calculation.
4. History list updates.

Navigation Flow 4: Clearing All History

1. User goes to History.
2. User taps Clear All.
3. Confirmation dialog appears.

4. User taps Cancel to keep history, or Clear All to delete everything.
5. If confirmed, Room deletes all saved history records.
6. Empty history message appears.

Navigation Flow 5: Starting Over

1. User returns to Calculator.
2. User taps Clear All.
3. App resets the calculator fields.
4. User can begin a new calculation.
5. Technical Architecture

TaxIt uses MVVM architecture.

MVVM means:

- View: Fragment and XML layouts
- ViewModel: Holds screen state and logic
- Model/Repository: Handles data from API and database

Calculator flow:

CalculatorFragment

→ CalculatorViewModel

→ TaxRepository

→ Ziptax API and Room database

History flow:

HistoryFragment

→ HistoryViewModel

→ TaxRepository

→ Room database

6. Special Notes

To run the project:

1. Open the project in Android Studio.
2. Add the Ziptax API key to local.properties.
3. Build and run the app on an emulator or Android device.

local.properties setup:

ZIPTAX_API_KEY=your_real_ziptax_key_here

Permissions Needed

The app requires internet access because it calls the Ziptax API.

Android permission:

android.permission.INTERNET

Known Issues / Limitations

1.API key security

The API key is stored in local.properties and passed into BuildConfig. This is good for a class project/demo, but it is not fully secure for a deployment Android app because APK contents can be inspected.

2. Ziptax account required

The app requires a valid Ziptax API key. Without it, tax lookup will fail.

3.Future Improvements

Future versions of TaxIt could include:

- Backend proxy server to protect API key
- Full street-address tax lookup for more accurate tax rates
- Separate discount fields saved in History
- Export/share calculation history
- Receipt or invoice PDF generation
- Dark mode
- User accounts and cloud sync